S.No: 6	Exp. Name: ***Developing a Single Page Application using RESTful APIs***	**Date: 2026-09-30**

Aim:

You are developing a Movie Management System for an online streaming platform. The application allows users to add new movies, update movie ratings, delete movies, and view all available movies.

The application behaves as a Single Page Application (SPA). Instead of reloading the page after every operation, it communicates with the Express.js server using RESTful APIs through JavaScript's `fetch()` method.

The MongoDB connection, Movie model, project configuration, HTML, CSS, and the basic JavaScript functions have already been provided. Your task is to complete the missing code in both **index.js** and **script.js** so that the application works correctly.

Your Tasks

Complete the missing code in **index.js** by implementing the following RESTful API routes.

1. Create Movie

Create a POST route at `/add-movie` that:

- Accepts book details ($title$, $director$, $year$, and $rating$) in JSON format.
- Stores the received data as a new **Movie** document in the database.

Responds with:

```
Movie added: <movie-title>
```

2.View Movies

Create a GET route at `/movies` that:

- Retrieves all movies from the database using `Movie.find()`.
- Returns the retrieved documents as JSON.
- If an error occurs, respond with status code 500 and an appropriate error message.

3. Update Movie rating

Create a PUT route at `/update-movie` that:

- Accepts $title$ and $newRating$ from the request body.
- Updates only the rating of the matching movie using the $set operator.
- If the update is successful, respond with:

```
Movie '<title>' rating updated successfully
```

- If no matching movie is found, respond with:

```
No movie found with title '<title>'
```

- If an error occurs, respond with status code `500`.

4. Delete Movie

Create a DELETE route at `/movies/:title` that:

- Deletes the movie whose title matches the route parameter.

- If the movie is deleted successfully, respond with: {"message": "Movie deleted successfully"}
- If no movie exists respond with: {"message": "Movie not found"}
- If an error occurs, respond with status code `500` and an appropriate error message.

Complete the missing code in **script.js** by completing `fetch()` requests for the following functions.

- **addMovie()** → Write a `fetch()` request that sends a `POST` request to `/add-movie` with the provided movie details in JSON format.
- **updateMovie()** → Write a `fetch()` request that sends a `PUT` request to `/update-movie` with the provided title and newRating in JSON format.
- **deleteMovie()** → Write a `fetch()` request that sends a `DELETE` request to `/movies/:title` by appending the movie title from the input field, encoded using `encodeURIComponent()`, to `/movies/`.
- **viewMovies()** → Write a `fetch()` request that sends a `GET` request to `/movies` and retrieves the movie list for display.

Run the application and verify that movies can be added, deleted, updated, and retrieved successfully.

Note: Do not modify the provided application setup. Write only the missing code in **index.js** and **script.js**.

Source Code:

express-app/index.js

```
const express = require('express');
const mongoose = require('mongoose');
const path = require('path');
const Movie = require('./models/movieModel');

const app = express();
app.use(express.json());
app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

let dbName = '';
let dbHost = '';
let dbPort = '';

mongoose.connect('mongodb://mongo:27017/libraryDB', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => {
  console.log('Database connected successfully!');

  const db = mongoose.connection;
  dbName = db.name;
  dbHost = db.host;
  dbPort = db.port;
})
.catch((err) => {
  console.error('Database connection failed:', err.message);
});

app.get('/ui', (req, res) => {
  res.render('index');
});

// Write POST /add-movie
app.post('/add-movie',async(req,res)=>{
        try {
                const {title, director,year,rating}= req.body;
                const movie = new
Movie({title,director,year,rating});
                await movie.save();
                res.send(`Movie added: ${movie.title}`);
        }
```

```
        catch (err) {
                res.status(500).send('Error adding movie:
'+err.message);
        }
});


// Write GET /movies
app.get('/movies',async(req,res)=>{
        try {
                const movies = await Movie.find();
                res.json(movies);
        }
        catch (err){
                res.status(500).send('Error retrieving movies:
'+err.message);
        }
});

// Write PUT /update-movie
app.put('/update-movie',async (req,res)=>{
        try{
                const {title, newRating }=req.body;
                const updated=await Movie.findOneAndUpdate(
                        { title },
                        { $set: {rating: newRating }},
                        { new: true }
                );
                if (!updated) {
                        return res.send(`No movie found with
title '${title}'`);
                }
                res.send(`Movie '${title}' rating updated
successfully`);
        }
        catch (err) {
                res.status(500).send('Error updating movie:
'+err.message);
        }
});

// Write DELETE /movies/:title
app.delete('/movies/:title', async (req,res)=>{
        try {
                const deleted = await Movie.findOneAndDelete({
title: req.params.title });
```

```
                if (!deleted) {
                        return res.json({ message: "Movie not
found" });
                }
                res.json({ message: "Movie deleted successfully"
});
        }
        catch (err) {
                res.status(500).json({ message: 'Error deleting
movie: '+err.message });
        }
});

// -------------------- DATABASE DETAILS --------------------

app.get('/', (req, res) => {
  res.json({
    dbName,
    dbHost,
    dbPort
  });
});


// -------------------- SERVER --------------------

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

express-app/public/script.js

```
function hideAll() {
    document.querySelectorAll(".page").forEach(page =>
page.style.display = "none");
}

function showPage(id) {
    hideAll();
    document.getElementById(id).style.display = "block";
    document.getElementById("output").innerHTML = "";
}

function goHome() {
    hideAll();
    document.getElementById("home").style.display = "block";
    document.getElementById("output").innerHTML = "";
}


async function addMovie() {
        const response = await fetch('/add-movie', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            title: document.getElementById("title").value,
            director: document.getElementById("director").value,
            year: Number(document.getElementById("year").value),
            rating:
Number(document.getElementById("rating").value)
        })
        });
    document.getElementById("output").innerHTML = await
response.text();
}

async function updateMovie() {
        const response=await fetch('/update-movie', {
                method: 'PUT',
                headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          title: document.getElementById("updateTitle").value,
          newRating:
Number(document.getElementById("newRating").value)
      })
        });
```

```
    document.getElementById("output").innerHTML = await
response.text();
}

// Write your code here for Delete Movie

async function deleteMovie() {
        const title=document.getElementById("deleteTitle").value;
        const response=await fetch('/movies/'
+encodeURIComponent(title), {
                method: 'DELETE'
        });
        const data=await response.json();
    document.getElementById("output").innerHTML = await
response.text();
}

async function viewMovies() {
    hideAll();
    document.getElementById("viewPage").style.display = "block";
// write the fetch to view
        const response=await fetch('/movies');
        const movies=await response.json();
    let table = `
    <table>
        <tr>
            <th>Title</th>
            <th>Director</th>
            <th>Year</th>
            <th>Rating</th>
        </tr>
    `;

    movies.forEach(movie => {

        table += `
        <tr>
            <td>${movie.title}</td>
            <td>${movie.director}</td>
            <td>${movie.year}</td>
            <td>${movie.rating}</td>
        </tr>
        `;

    });
```

```
    table += "</table>";
    document.getElementById("movieTable").innerHTML = table;
}
```